

RELAZIONE SERVIZIO OPEN SKY NETWORK

HOMEWORK 3

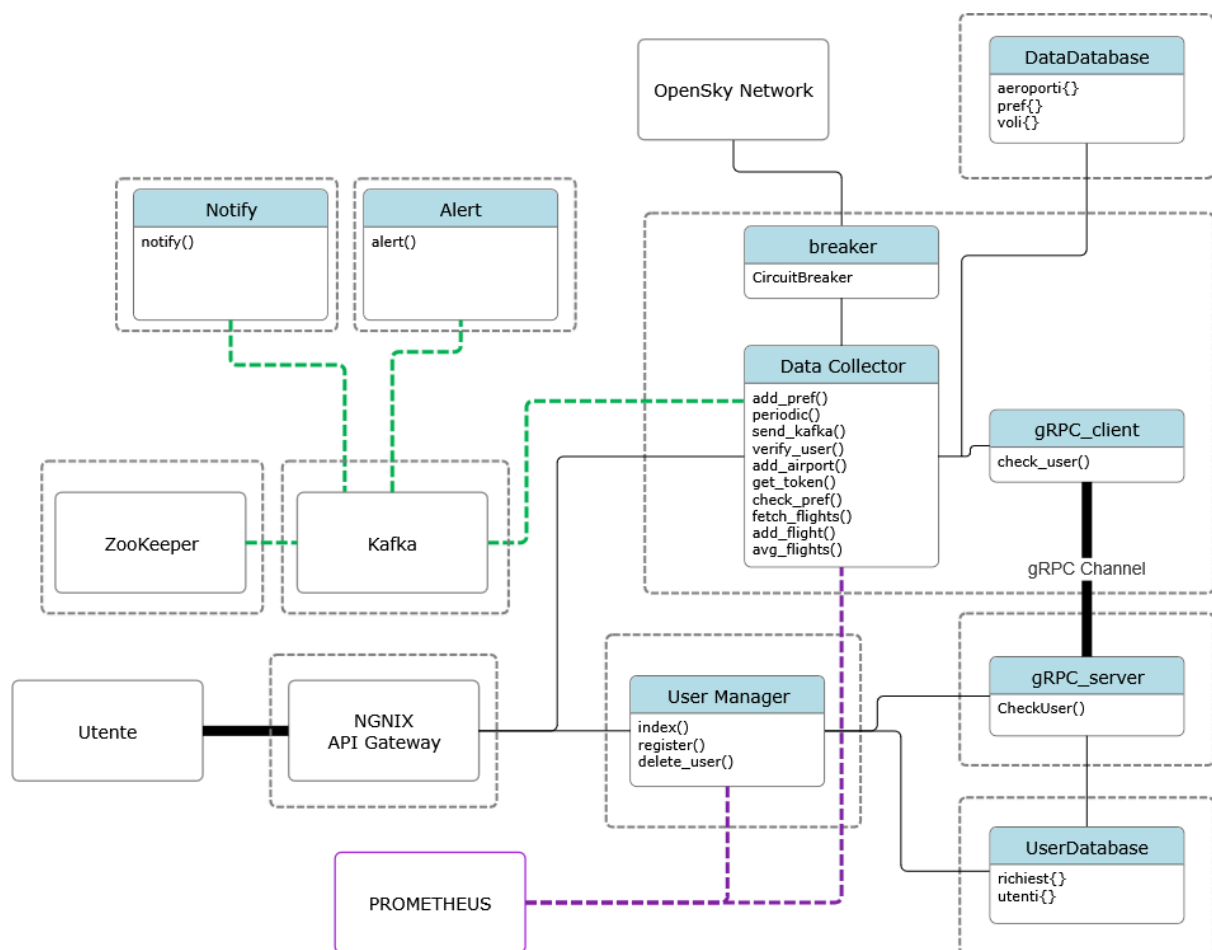
INTEGRAZIONE KUBERNETES & PROMETHEUS

Casaccio Agrippino 1000085471

Rizzo Alessio 1000082581

L'obiettivo è stato quello di ampliare il sistema dockerizzato a microservizi descritto nelle relazioni precedenti, estendendo le funzionalità con un meccanismo di white-boxing monitoring, per poi essere deployato su Kubernetes.

Per poter realizzare la prima parte del lavoro è stato introdotto un altro servizio, Prometheus; per la seconda parte, si è deciso di migrare l'intera struttura da Docker Compose a un cluster locale con kind, creando Deployment per ogni servizio.



AGGIORNAMENTO & NUOVI SERVIZI

1. Prometheus

Prometheus è un toolkit usato per il monitoraggio e funge da orchestratore delle metriche da valutare. In questo contesto, si occupa di interrogare periodicamente i due microservizi principali scelti: UserManager e DataCollector.

Si è deciso di sviluppare, per compattezza e poichè richiesto l'utilizzo di K8s, un unico file Manifest, definente il Server Prometheus, diviso in tre sezioni:

La sezione detta *Configuration* ne descrive il comportamento, ogni 5 secondi va ad interrogare i due endpoint target e raccoglie le metriche.

La sezione *Deployment*, esattamente come un deployment Kubernetes, creerà il Pod relativo, chiamato per semplicità prometheus e ne gestirà il ciclo di vita, montando la configurazione precedente come volume.

La sezione *Service* invece lo espone; in questo caso è possibile accedervi dall'esterno tramite porta 30090.

Le metriche ricevute verranno poi memorizzate con un timestamp e i label (nome del servizio e nome del nodo).

2. UserManager & DataCollector

User Manager e Data Collector sono i due microservizi, i primi ad essere stati sviluppati e il cuore di tutto il progetto. Per questo motivo, oltre al fatto che sono loro ad usare maggiormente i dati, ad essere scelti per il white-box monitoring di prometheus. Entrambi espongono due metriche, una GAUGE e una COUNTER:

- *user_manager_rich*: un COUNTER che incrementa ad ogni invocazione del “/register”; serve dunque a misurare il traffico in ingresso e quanti utenti si registrano al servizio.
- *user_db_latenza*: un GAUGE che rappresenta il tempo impiegato per completare le varie operazioni di SELECT e INSERT sul database user_db. Viene usato sempre nella route “/register”.
- *data_collector_msg_kafka*: un COUNTER dedicato al numero totale di messaggi Kafka correttamente inviati, registrati inoltre con il topic relativo, in questo caso *to-alert-system*. Adoperato nella funzione send_kakfa().
- *data_collector_fetch_time*: un GAUGE che misura il tempo impiegato a scaricare i dati dall'API di OpenSky Network, il quale risulta più o meno lungo a seconda dell'aeroporto scelto e della giornata. Adoperato nella funzione fetch_flight().

CONSIDERAZIONI SU KUBERNATES

Data la consegna e per mantenere una coerenza tra le varie componenti, si è deciso di creare i Deployment ed i Pod anche per i rimanenti microservizi:

Per quanto riguarda i DataBase, nel deployment è stata aggiunta una *ConfigMap* per poterli inizializzare all'avvio del Pod e a tal scopo è stato inserito al loro interno il

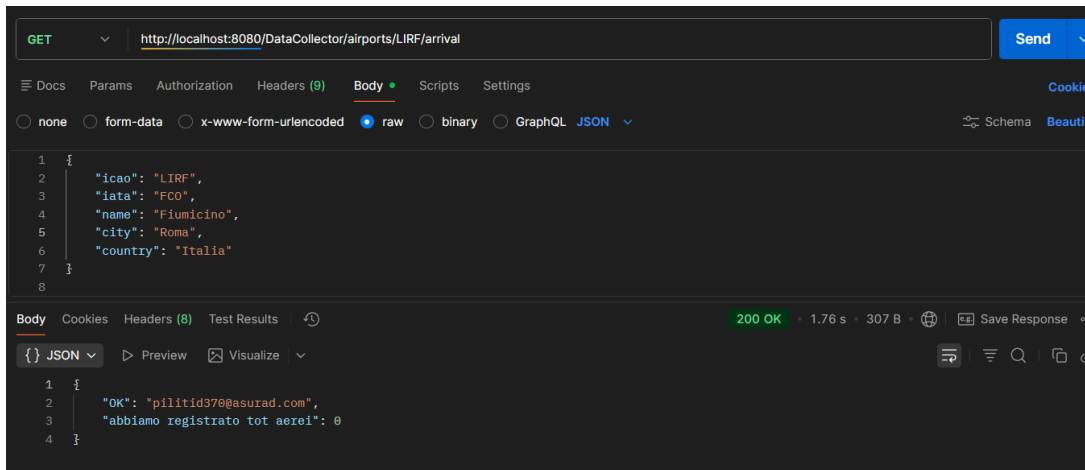
codice sql relativo (sia per userdatabase che per datadatabase). Si è anche aggiunto un *PersistentVolumeClaim* per la persistenza dati, con 1Gb di storage default, e impostata una strategy *Recreate* per poter essere ricreati correttamente con nuovi aggiornamenti ed evitando problemi di concorrenza sull'accesso.

Anche l'Api Gateway NGNIX presenta all'interno del file YAML una *ConfigMap* contenente il file di configurazione, permettendo così anche di eventualmente modificare le regole di routing senza dover ricostruire l'immagine del container. Per le richieste HTTP, i servizi sono ora identificati tramite i rispettivi Service Kubernetes.

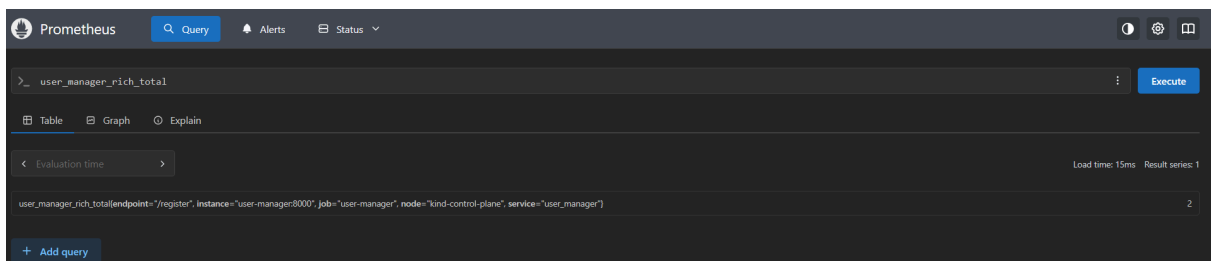
I microservizi Alert e Notify sono stati deployati senza nessuna aggiunta degna di nota, così come l'smtp-relay usato per mandare le email; si è fatta comunque attenzione a riportare correttamente i parametri Kafka. Come conseguenza si è effettuato il Deployment anche di Kafka e di Zookeeper. In tal modo per ogni microservizio precedentemente realizzato si è creato un Pod dedicato.

Per poterlo eseguire è necessario inserire le credenziali di OpenSky Network sottoforma di file denominato configuration.json nella directory DataCollector.

L'esecuzione è avvenuta mediante riga di comando nella root principale, da prima creato il cluster utilizzando Kind, create le immagini e caricate all'interno mediante "kind load docker-image", applicati i manifest YAML e avviato i Pod. Si è poi verificato lo stato dei Pod, il loro funzionamento mediante Postman e Prometheus esponendo le relative porte.



un esempio sul funzionamento mediante Postman



un esempio delle metriche sul servizio UserManager